TOB-MAQ

COLLECTION
CRÉATION WEB

LIVRE DE FORMATION · AVANCÉ

Développement web full-stack

Concevoir une application web complète : interface, API, base de données, sécurité et mise en ligne.



Gratuit | Avancé | 16 h de formation | 3 chapitres

By OLUWATOBI

BIENVENUE

Ce que tu vas apprendre

Un développeur full-stack sait construire une application complète, de l'écran jusqu'à la base de données. Ce livre t'apprend l'architecture, la création d'API et la mise en production sécurisée.

SOMMAIRE

01 **Architecture d'une application web** — 40 min
Architecture et API

02 **Créer une API REST avec Node.js et Express** — 50 min
Architecture et API

03 **Sécuriser et mettre en ligne** — 45 min
Mise en production

✓ **Quiz d'auto-évaluation**

Aa **Glossaire, exercices et mémo**

FICHE DE LA FORMATION

Développement web full-stack

NIVEAU
Avancé

DURÉE
16 heures

DOMAINE
Création web

ACCÈS
Gratuit

OUTILS
Visual Studio Code, Node.js

À LA FIN DE CE LIVRE, TU SAURAS

- Comprendre l'architecture d'une application web
- Créer une API REST avec Node.js
- Sécuriser et déployer l'application

COMMENT UTILISER CE LIVRE
Lis un chapitre à la fois, réalise l'exercice proposé, puis vérifie tes acquis avec le quiz. Les leçons complètes, les jeux et l'examen final t'attendent sur l'application TOB-MAQ.

CERTIFICAT
Réussis l'examen final de la formation (10/10) sur l'application pour obtenir ton certificat de mérite à ton nom.

01

ARCHITECTURE ET API

Architecture d'une application web

OBJECTIF Comprendre comment le front-end, le back-end et la base de données travaillent ensemble.

Une application web complète a trois couches : le **front-end** (ce que voit l'utilisateur), le **back-end** (la logique et la sécurité, sur le serveur) et la **base de données** (la mémoire). Elles communiquent par une **API**.

1 **Front-end**
HTML, CSS, JavaScript (ou React, Vue) : affichage, formulaires, interactions.

2 **Back-end**
Node.js, Python ou PHP : reçoit les requêtes, vérifie les droits, applique les règles métier.

3 **Base de données**
Relationnelle (PostgreSQL, MySQL, SQLite) pour des données structurées : utilisateurs, commandes.

4 **API REST**
Des adresses et des verbes : GET /produits lit, POST /commandes crée, PUT modifie, DELETE supprime.

5 **Le parcours d'une requête**
Le navigateur envoie, le serveur vérifie et interroge la base, puis renvoie du JSON que l'interface affiche.

SI TU DÉBUTES
Retiens la règle d'or : on ne fait jamais confiance au navigateur, tout est revérifié sur le serveur.

BON À SAVOIR
Commence simple (un serveur, une base) ; les architectures complexes ne se justifient qu'avec beaucoup d'utilisateurs.

EXEMPLE CONCRET

TOB-MAQ elle-même : une interface en HTML et JavaScript, un serveur Node.js avec Express, une base SQLite et une API sous /api.

À RETENIR

- Front-end, back-end, base de données.
- Une API relie le tout.
- GET, POST, PUT, DELETE.
- Toujours vérifier côté serveur.

À TOI DE JOUER

Dessine l'architecture d'une application de réservation de salon de coiffure : écrans, routes de l'API et tables de la base.

02

ARCHITECTURE ET API

Créer une API REST avec Node.js et Express

OBJECTIF Construire une API qui lit et enregistre des données, avec validation et gestion des erreurs.

Express est le cadre le plus utilisé pour créer des serveurs avec Node.js. Quelques lignes suffisent pour une API ; la qualité professionnelle vient de la **validation**, des **codes d'erreur** clairs et de la **sécurité**.

1 **Initialise**

npm init -y puis npm install express ; crée server.js.

2 **Première route**

app.get('/api/produits', (req, res) => res.json(produits)); renvoie la liste en JSON.

3 **Recevoir des données**

app.use(express.json()) puis app.post('/api/produits', …) lit req.body.

4

Valide

Vérifie chaque champ (nom non vide, prix nombre positif) et réponds 400 avec un message clair sinon.

5

Codes de réponse

200 OK, 201 créé, 400 donnée invalide, 401 non connecté, 404 introuvable, 500 erreur serveur.

SI TU DÉBUTES

Teste tes routes avec un outil comme Thunder Client (extension VS Code) ou curl.

BON À SAVOIR

Utilise des requêtes SQL préparées (avec paramètres) : elles empêchent les injections SQL.

EXEMPLE CONCRET

L'API d'une boutique : liste des produits, ajout par l'administrateur, commandes validées côté serveur avec le vrai prix (jamais celui envoyé par le navigateur).

À RETENIR

- Express : routes GET et POST.
- express.json() pour lire le corps.
- Valider chaque champ.
- Codes HTTP clairs.
- Requêtes SQL préparées.

À TOI DE JOUER

Crée une API de gestion de rendez-vous : lister, créer (avec validation), annuler ; teste chaque route.

03

MISE EN PRODUCTION

Sécuriser et mettre en ligne

OBJECTIF Préparer une application pour la production : secrets, sécurité, hébergement et suivi.

Mettre en ligne, c'est exposer son application à tout Internet. Les bonnes pratiques de base protègent les utilisateurs : **HTTPS**, **mots de passe hachés**, **secrets hors du code** et **sauvegardes**.

1 **Variables d'environnement**
process.env.DATABASE_URL, process.env.API_KEY : configurées chez l'hébergeur, jamais dans le dépôt.

2 **Mots de passe**
Toujours hachés (scrypt, bcrypt, argon2) : on ne stocke jamais un mot de passe en clair.

3 **Cookies et sessions**
Cookies HttpOnly, Secure et SameSite pour les sessions.

4 **Héberger**
Render, Railway ou un VPS : le dépôt GitHub est relié, chaque mise à jour redéploie l'application.

5 **Surveiller et sauvegarder**
Journaux d'erreurs, route /health, sauvegarde régulière de la base de données.

SI TU DÉBUTES
Ne mets jamais une clé secrète ou un mot de passe dans le code envoyé sur GitHub : utilise des variables d'environnement.

BON À SAVOIR
Ajoute des limites de requêtes (rate limiting) sur la connexion pour bloquer les attaques par force brute.

EXEMPLE CONCRET

Une application de réservation est déployée sur Render : clés dans les variables d'environnement, base sauvegardée chaque jour, HTTPS automatique.

À RETENIR

- Secrets dans l'environnement.
- Mots de passe hachés.
- Cookies sécurisés.
- Déploiement relié à GitHub.
- Journaux et sauvegardes.

À TOI DE JOUER

Fais la liste de contrôle « mise en production » de ton projet : 10 points de sécurité et de fiabilité à cocher.

AUTO-ÉVALUATION

Quiz : vérifie tes acquis

Réponds sans regarder les chapitres, puis compare avec les réponses en bas de page.

1. Où doit se faire la vérification des droits ?

A Sur le serveur (back-end) B Uniquement dans le navigateur C Nulle part

2. Quel format échangent souvent l'API et l'interface ?

A JSON B MP3 C DOCX

3. Pourquoi recalculer le prix sur le serveur ?

A Le navigateur peut envoyer un prix modifié B Pour aller plus vite C Ce n'est pas utile

4. Comment stocker les mots de passe des utilisateurs ?

A Hachés (bcrypt, scrypt, argon2) B En clair C Dans un fichier texte

5. Où vérifier les droits d'un utilisateur ?

A Sur le serveur B Seulement dans le navigateur C Nulle part

6. Quel code HTTP signifie « créé » ?

A 201 B 404 C 500

7. Comment stocker les mots de passe ?

A Hachés (bcrypt, scrypt) B En clair C Dans un fichier texte

8. Quels attributs pour un cookie de session sûr ?

A HttpOnly, Secure, SameSite B Public et sans date C Aucun

Réponses : 1-A · 2-A · 3-A · 4-A · 5-A · 6-A · 7-A · 8-A

LES MOTS DU MÉTIER

Glossaire

Front-end	Partie de l'application affichée dans le navigateur.
Back-end	Partie serveur : logique, sécurité, données.
API REST	Interface d'échange organisée en adresses et verbes HTTP.
JSON	Format de données échangées entre serveur et interface.
Code HTTP	Nombre qui indique le résultat d'une requête (200, 404…).
Hachage	Transformation irréversible d'un mot de passe.
Variable d'environnement	Réglage secret configuré chez l'hébergeur.
Déploiement	Mise en ligne de l'application.

PASSE À LA PRATIQUE

Projet et mémo

PROJET DE FIN DE LIVRE

Application de réservation en ligne

Développe une application de réservation (comptes, créneaux, réservations) avec API, base de données et mise en ligne.

CE QUE TU DOIS LIVRER

- Code sur GitHub
- Application en ligne
- Liste de contrôle de sécurité remplie

MÉMO À GARDER

- [ ] Ne jamais faire confiance au navigateur.
- [ ] Valider chaque donnée reçue.
- [ ] Requêtes SQL préparées.
- [ ] Mots de passe hachés.
- [ ] Secrets hors du code.
- [ ] Sauvegardes et journaux.

TOB-MAQ

By OLUWATOBI

TOB-MAQ est l'académie en ligne des métiers créatifs et du numérique au Bénin : formations, jeux, ressources et certificats, pour les 15–40 ans.

Continue sur l'application

Leçons complètes, jeux, examen final et certificat de mérite. Scanne pour l'installer.

TOB-MAQ.ONRENDER.COM